핵심 구현 — 동시 예약 방지와 포인트 이중 사용 방지

1) 예약 (ReservationService.reserve)

```
     * 좌석 예약.
     * 1) 회원 행을 잠가 같은 회원의 동시 예약이 포인트를 이중으로 쓰지 못하게 하고
     * 2) 예약을 1시간 슬롯으로 쪼개 넣어 (seat_id, slot_start) 유니크 제약이 겹치는 예약을 DB 단에서 거부하게 한다.
     */
    @Transactional
    public Reservation reserve(Long memberId, Long seatId, LocalDateTime startAt, int hours, LocalDateTime now) {
        // 회원 락을 트랜잭션의 첫 문장으로 잡는다. MySQL 기본 격리(REPEATABLE READ)에서는 첫 일반 SELECT 때
        // 읽기 스냅샷이 만들어지므로, 락보다 먼저 다른 조회를 하면 앞 트랜잭션이 막 커밋한 사용 내역이 안 보여
        // 잔액 검증을 잘못 통과할 수 있다. (격리 수준도 READ COMMITTED로 맞춰 두었다 - application.yml)
        Member member = members.findByIdForUpdate(memberId)
                .orElseThrow(() -> new IllegalArgumentException("회원을 찾을 수 없습니다"));
        CafePolicy policy = policy();
        Seat seat = seats.findById(seatId).filter(Seat::isActive)
                .orElseThrow(() -> new IllegalArgumentException("이용할 수 없는 좌석입니다"));
        checkSeatAllowed(seat, member);
        Zone zone = zones.findById(seat.getZoneId()).orElseThrow();
        Branch branch = branches.findById(zone.getBranchId()).orElseThrow();
        validate(startAt, hours, now, policy, branch);

        int price = hours * zone.getPricePointPerHr();
        Reservation reservation = reservations.save(
                new Reservation(memberId, seatId, startAt, startAt.plusHours(hours), price, newCode(), now));

        List<ReservationSlot> rows = new ArrayList<>();
        for (int i = 0; i < hours; i++) {
            rows.add(new ReservationSlot(reservation.getId(), seatId, startAt.plusHours(i)));
        }
        try {
            slots.saveAllAndFlush(rows);
        } catch (DataIntegrityViolationException e) {
            throw new SeatTakenException("이미 예약된 시간이 포함되어 있습니다");
        }
        points.use(memberId, reservation.getId(), price, now);
```

```
        return reservation;
    }
```

2) 슬롯 테이블과 유니크 제약 (V1__init.sql)

```
-- 예약 1건을 1시간 단위로 쪼갠 행. (seat_id, slot_start) 유니크가 이중 예약을 DB에서 막는다.
-- 취소/노쇼 시 이 행들을 삭제해 자리를 다시 연다.
CREATE TABLE reservation_slot (
    id             BIGINT      NOT NULL AUTO_INCREMENT PRIMARY KEY,
    reservation_id BIGINT      NOT NULL,
    seat_id        BIGINT      NOT NULL,
    slot_start     DATETIME    NOT NULL,
    CONSTRAINT uk_slot UNIQUE (seat_id, slot_start),
    CONSTRAINT fk_slot_rsv FOREIGN KEY (reservation_id) REFERENCES reservation (id)
);
```

핵심 — ① 회원 행을 **트랜잭션 첫 문장**에서 잠가(비관적 락) 같은 회원의 동시 예약이 포인트를 이중으로 쓰지 못하게 하고, ② 예약을 1시간 슬롯 행으로 쪼개 `(seat_id, slot_start)` 유니크 제약이 겹치는 예약을 DB가 거부하게 한다. ③ 격리 수준은 READ COMMITTED.

증명한 시험

시험	검증 내용
같은 좌석·같은 시간에 20명이 동시에 예약	성공 1, 거부 19, 슬롯 2개·예약 1건만 남음
겹치는 예약 12건 혼합(10·11·12시 시작, 각 2시간)	같은 좌석·같은 시간이 두 번 잡힌 곳 0건, 성공 예약 수 × 2 = 슬롯 수(실패한 예약은 슬롯을 남기지 않음)
정책 위반 예약	운영시간 밖·지난 시간·정시 아님·최대 시간 초과 거부, 21~23시는 허용
같은 회원이 여러 좌석을 동시에 예약	잔액 2,000P로 5좌석 동시 예약 → 1건 성공·4건 포인트 부족, 잔액 0(음수 없음)